

CMP1203 – Fundamentos de Programação Orientada a Objetos

Prof. Me Alexandre Cláudio de Almeida



**PUC
GOIÁS**

Classes Abstratas e Interfaces



Introdução

- Herança: poderosa ferramenta (superclasse e subclasse)
- Planejamento entre classes ancestrais e herdeiras **não é simples**
- Nem sempre a classe ancestral deve ser instanciada
 - apenas descrever os campos e métodos que as classes herdeiras devem implementar
- A classe ancestral dita **o que** deve ser feito, mas não necessariamente **como** deve ser feito

Exemplo

- Pessoa, Funcionário e ChefeDeDepartamento:
 - não faz sentido instanciar uma Pessoa
 - no entanto eliminar a classe transferindo para Funcionário a tarefa de ser a superclasse, não funciona.
 - Várias aplicações utilizam Pessoa

Classes Abstratas



Métodos Abstratos

- São somente declarados (nome, modificadores, tipo de retorno e lista de argumentos), não tendo um corpo com comandos
- Se uma classe tiver algum método abstrato, a classe também deverá ser declarada com o modificador **abstract**
- Uma classe que herde de uma classe abstrata, deverá, obrigatoriamente, implementar todos os métodos declarados como abstratos na classe ancestral

Robô Abstrato

```
abstract class RoboAbstrato {  
    private String nome;  
    private int posicaoXAtual, posicaoYAtual;  
    private short direcaoAtual; //em graus  
    RoboAbstrato(String n, int px, int py, short d) {  
        nome = n;  
        posicaoXAtual = px;  
        posicaoYAtual = py;  
        direcaoAtual = d;  
    }  
  
    public void move() {  
        move(1);  
    }  
  
    public abstract void move(int passos);
```

```
17  
18  
19  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32  
33  
34  
35  
    public void moveX(int passos) {  
        posicaoXAtual += passos;  
    }  
    public void moveY(int passos) {  
        posicaoYAtual += passos;  
    }  
    public void mudaDirecao(short novaDirecao) {  
        direcaoAtual = novaDirecao;  
    }  
    public short qualDirecaoAtual() {  
        return direcaoAtual;  
    }  
  
    public String toString() {  
        String resultado;  
        resultado = "Nome do Robô: " + nome + "\nPosição:  
        (" + posicaoXAtual + "," + posicaoYAtual +  
        ")\nDireção: " + direcaoAtual + "\n";  
        return resultado;  
    }  
}
```

Robô Simples

```
1  class RoboSimples extends RoboAbstrato {  
2      RoboSimples(String n, int px, int py, short d) {  
3          super(n, px, py, d);  
4      }  
5  
6      public void move(int passos) {  
7          switch (qualDirecaoAtual()) {  
8              case 0: moveX(passos); break;  
9              case 90: moveY(passos); break;  
10             case 180: moveX(-passos); break;  
11             case 270: moveY(-passos); break;  
12         }  
13     }  
14 }
```


Robô Bateria

```
1 class RoboBateria extends RoboAbstrato {
2     private long energia;
3     RoboBateria(String n, int px, int py, short d, long e)
4     {
5         super(n, px, py, d);
6         energia = e;
7     }
8     public void move(int passos) {
9         long energiaASerGasta = passos * 10;
10        if (energiaASerGasta <= energia) {
11            switch (qualDirecaoAtual()) {
12                case 0:      moveX(passos); break;
13                case 45:     moveX(passos);
14                           moveY(passos); break;
15                case 90:     moveY(passos); break;
16                case 135:    moveY(passos);
17                           moveX(-passos); break;
18                case 180:    moveX(-passos); break;
19                case 225:    moveX(-passos);
20                           moveY(-passos); break;
21                case 270:    moveY(-passos); break;
22                case 315:    moveY(-passos);
23                           moveX(passos); break;
24            }
25            energia -= energiaASerGasta;
26        }
27    }
28 }
```

```
28 public String toString() {
29     String resultado;
30     resultado = super.toString() + "\n";
31     resultado += "Energia: " + energia + "\n";
32     return resultado;
33 }
34 }
```

Demonstração do Robô

```
1  class RoboDemo {  
2      public static void main(String[] args) {  
3          RoboSimples r2d2;  
4          RoboBateria c3po;  
5          RoboAbstrato ghost;  
6          r2d2 = new RoboSimples("R2D2", 0, 0, (short)90);  
7          c3po = new RoboBateria("C3PO", 0, 0, (short)90, 111);  
8          //ghost = new RoboAbstrato("GHOST", 0, 0, (short)180);  
9          r2d2.move(10);  
10         r2d2.mudaDirecao((short)180);  
11         r2d2.move();  
12         r2d2.move();  
13         System.out.println(r2d2);  
14         c3po.move(10);  
15         c3po.mudaDirecao((short)180);  
16         c3po.move();  
17         c3po.move();  
18         System.out.println(c3po);  
19     }  
20 }
```

Nome do Robô: R2D2
Posição: (-2,10)
Direção: 180

Nome do Robô: C3PO
Posição: (-1,10)
Direção: 180

Energia: 1

Interfaces



O que é uma Interface?

- Se a classe não tiver nenhum método não-abstrato, podemos criar uma interface
- Uma interface não pode ser instanciada.
- Todos os métodos na interface são implicitamente abstract e public e são vazios.
- Se houver campos, serão implicitamente considerados como static e final (devem ser inicializados na declaração)

Classe Abstrata X Interfaces

- Uma classe herdeira somente pode herdar de uma única classe (abstrata ou não).
- Qualquer classe pode implementar várias interfaces simultaneamente.
- Interface é uma forma simplificada de implementação de herança múltipla em Java.
- Como todos os campos de uma interface devem ser declarados como `static` e `final`, podemos escrever interfaces que tenham somente campos (biblioteca de constantes)

Ponto 2D

```
public class Ponto2D {  
    private double x;  
    private double y;  
    public Ponto2D(double x, double y) {  
        setX(x);  
        setY(y);  
    }  
    public double getX() {  
        return x;  
    }  
    public void setX(double x) {  
        this.x = x;  
    }  
    public double getY() {  
        return y;  
    }  
    public void setY(double y) {  
        this.y = y;  
    }  
    public String toString() {  
        return "Eu sou um ponto BIDIMENSIONAL (2D)\n" +  
            "Minha coordenada X = " + getX() + "\n" +  
            "Minha coordenada Y = " + getY() + "\n";  
    }  
}
```

Objeto Geométrico

```
1 //os métodos são implicitamente abstract e public
2 //não podem ter construtores
3 interface ObjetoGeometrico {
4     Ponto2D centro();
5     double calculaArea();
6     double calculaPerimetro();
7 }
```

Círculo

```
class Circulo implements ObjetoGeometrico {
    private Ponto2D centro;
    private double raio;

    Circulo(Ponto2D centro, double raio) {
        this.centro = centro;
        this.raio = raio;
    }

    public Ponto2D centro() {
        return centro;
    }

    public double calculaArea() {
        return Math.PI * raio * raio;
    }

    public double calculaPerimetro() {
        return 2.0 * Math.PI * raio;
    }

    public String toString() {
        return "Circulo com centro em " + centro + " e  
raio " + raio;
    }
}
```


Retângulo

```
class Retangulo implements ObjetoGeometrico {
    private Ponto2D primeiroCanto, segundoCanto;

    Retangulo (Ponto2D primeiroCanto, Ponto2D segundoCanto) {
        this.primeiroCanto = primeiroCanto;
        this.segundoCanto = segundoCanto;
    }

    public Ponto2D centro() {
        double coordX = (primeiroCanto.getX() + segundoCanto.getX()) / 2.;
        double coordY = (primeiroCanto.getY() + segundoCanto.getY()) / 2.;
        return new Ponto2D(coordX, coordY);
    }

    public double calculaArea() {
        double ladoX = Math.abs(primeiroCanto.getX() - segundoCanto.getX());
        double ladoY = Math.abs(primeiroCanto.getY() - segundoCanto.getY());
        return ladoX * ladoY;
    }

    public double calculaPerimetro() {
        double ladoX = Math.abs(primeiroCanto.getX() - segundoCanto.getX());
        double ladoY = Math.abs(primeiroCanto.getY() - segundoCanto.getY());
        return (2 * ladoX) + (2 * ladoY);
    }

    public String toString(){
        return "Retangulo com cantos " + primeiroCanto + " e " + segundoCanto;
    }
}
```

Demonstração

```
class ObjetoGeometricoDemo {
    public static void main (String[] args) {
        Circulo c1, c2, c3;
        c1 = new Circulo(new Ponto2D(0,0), 100);
        c2 = new Circulo(new Ponto2D(-1,-1), 1);
        c3 = new Circulo(new Ponto2D(10,8), 0);
        Retangulo r1 = new Retangulo(new Ponto2D(-2, -2), new Ponto2D(2, 2));
        Retangulo r2 = new Retangulo(new Ponto2D(-100, -1), new Ponto2D(100, 1));
        Retangulo r3 = new Retangulo(new Ponto2D(0, 0), new Ponto2D(0, 0));
        imprimeTudo(c1);
        imprimeTudo(c2);
        imprimeTudo(c3);
        imprimeTudo(r1);
        imprimeTudo(r2);
        imprimeTudo(r3);
    }

    private static void imprimeTudo(ObjetoGeometrico og) {
        System.out.println(og);
        System.out.println("Perimetro: " + og.calculaPerimetro());
        System.out.println("Area: " + og.calculaArea());
        System.out.println();
    }
}
```

Dúvidas?

